

Engenharia de Serviços em Rede

Trabalho Prático N.º2

Serviço Over the Top para Entrega de Multimédia

MEI - 2023/2024

André Nunes
A85635



Afonso Bessa
PG53597



Francisco Claudino
PG50380



Grupo 66



Universidade do Minho

Conteúdo

1	Introdução	4
2	Arquitetura da Solução	5
3	Estrutura do Projeto	8
4	Especificação dos Protocolos	11
4.0.1	Diagrama de Pedido de Flood para Ligações Unique Way	14
4.0.2	Diagrama de Stream	15
4.0.3	Diagrama de Paragem da Stream	15
4.0.4	Diagrama do Still Alive	16
5	Implementação	17
6	Testes e Resultados	22
7	Exemplo de Execução	30
8	Conclusões e Trabalho Futuro	32

Lista de Figuras

2.1	Diagrama da Arquitetura da Rede	6
3.1	Estrutura do Projeto	8
3.2	Vídeo da Stream	9
4.1	Estrutura de uma mensagem do tipo <i>Flood</i>	11
4.2	Flags de uma mensagem do tipo <i>Flood</i> do Nodo	12
4.3	Estrutura de uma mensagem do tipo <i>Still Alive</i>	12
4.4	Estrutura de uma Mensagem do tipo <i>Stream</i> para o RP	13
4.5	Flags de uma Mensagem do tipo <i>Stream</i> para o RP, Cliente e Nodo	13
4.6	Flags de uma Mensagem do tipo <i>Stream</i> para o Server	13
4.7	Estrutura de uma mensagem do tipo <i>Play</i>	13
4.8	Diagrama de Pedido de Flood	14
4.9	Diagrama de Pedido de Flood para ligações Unique Way	14
4.10	Diagrama de Stream	15
4.11	Diagrama da Paragem da Stream	15
4.12	Diagrama do Still Alive	16
5.1	Exemplo da Estrutura do <i>bootstrap.txt</i>	18
5.2	Exemplo do Ficheiro <i>bootstrap.txt</i>	18
5.3	Resultado da Execução do <i>bootstrap.txt</i>	18
6.1	Topologia	22
6.2	Flood só com Nodos - a usar: n3 (cima) e n4 (baixo)	23
6.3	Announcement do Servidor	24

6.4	Estabelecimento da Melhor Rota	24
6.5	Distribuição da Stream pelos Nodos	25
6.6	Condicionamento da ligação	26
6.7	Alteração da rota devido ao condicionamento da ligação	26
6.8	Paragem da Stream	27
6.9	Monitorização da Rede	28
6.10	Unique Way	28
6.11	Adição e remoção de nodos 1	29
6.12	Adição e remoção de nodos 1	29
7.1	Diagrama de Sequência representativo do Flooding	30
7.2	Diagrama de Sequência representativo da Stream	31

Capítulo 1

Introdução

O objetivo primordial deste trabalho é desenvolver um sistema de transmissão de vídeo em tempo real a partir de um servidor de conteúdos para vários clientes. Para alcançar este propósito, será designado um **Rendezvous Point**, encarregado de receber o conteúdo via *unicast* para distribuição, difundindo-o por uma série de nós que atuam como intermediários. Estes nós formam uma árvore de distribuição partilhada, cuja criação e manutenção devem ser otimizadas para efetuar a entrega dos conteúdos de maneira eficiente, minimizando a latência e a largura de banda necessária.

Com esse propósito, foi implementada uma rede *overlay* aplicacional, que possibilita o reenvio de dados através de um conjunto de nodos em rede atuando como intermediários. Esta rede possui a capacidade de se adaptar a possíveis falhas, como atrasos na ligação entre determinados nodos ou a inatividade de alguns destes.

Neste relatório, no Capítulo 2 é apresentada a solução da arquitetura adotada onde é explicado como é que estes componentes comunicam uns com os outros. De seguida, no capítulo 3 é apresentada a divisão do código e todos os ficheiros que constituem o projeto. No Capítulo 4 é apresentada a especificação dos protocolos, onde a constituição das várias mensagens trocadas entre os componentes são apresentadas e explicadas. O Capítulo 5 descreve como a solução foi implementada, que decisões foram tomadas e como funcionam os diversos algoritmos aplicados. No Capítulo 6 apresenta os testes feitos à rede e os seus resultados. Por fim, no Capítulo 7 mostramos de forma muito simples como o Flooding e a Transmissão de Stream está retratada no nosso projeto.

Capítulo 2

Arquitetura da Solução

A solução desenvolvida é constituída por cinco componentes (Figura 2.1):

1. **Server**
2. **Node**
3. **Client**
4. **BootStrapper**
5. **Rendezvous Point**

O **BootStrapper**, como o nome sugere, atua como um *bootstrapper* na rede, permitindo a descoberta dos vizinhos dos nós e dos servidores através de uma conexão TCP.

O **Server** é responsável por fornecer a *stream* ao RP através de UDP.

O **Node** representa um nodo na rede que comunica com todos os componentes através de TCP para a criar e gerir a rede *overlay*. Este também comunica com o RP, o Client e outros nodos por UDP, de modo a fazer a distribuição da *stream*. Adicionalmente, este componente comunica com o *bootstrapper* por TCP para saber que nodos são seus vizinhos e com os outros nodos através do mesmo protocolo para a gestão e monitorização da rede *overlay*.

O **Client** é a aplicação do cliente que vai receber a *stream* da rede por UDP. Este comunica com o seu nodo através de TCP.

O **Rendezvous Point** comunica com o Node no *flooding*, na monitorização da rede e no estabelecimento de rotas. São também estabelecidas comunicações entre este e o Server.

A figura seguinte mostra como todos os componentes comunicam entre si e com quais protocolos usam na sua comunicação:

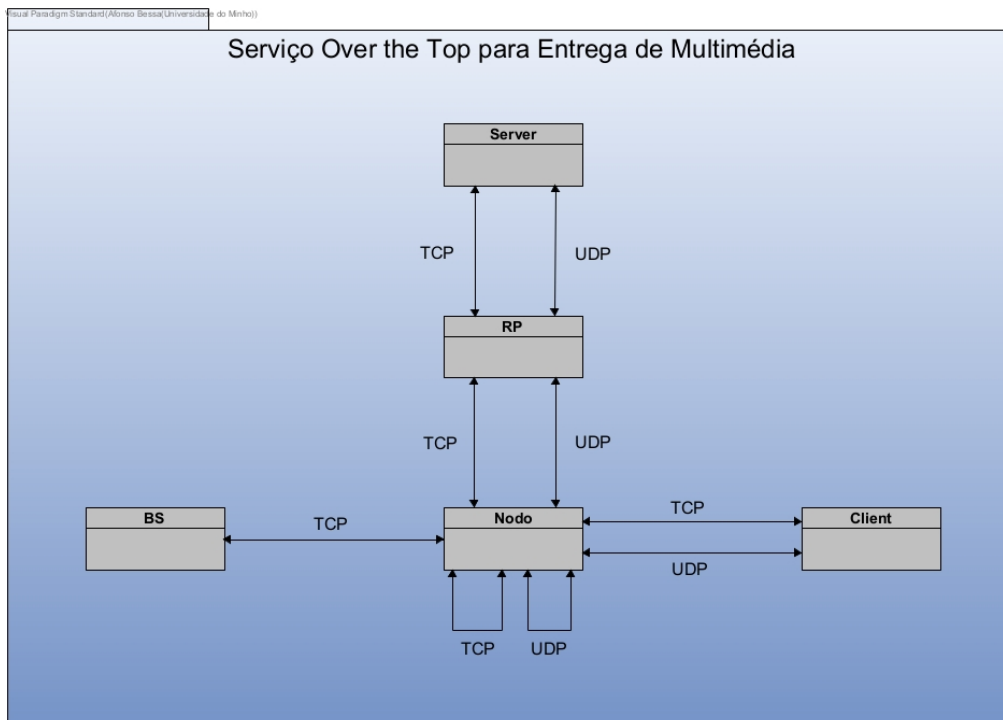


Figura 2.1: Diagrama da Arquitetura da Rede

Razões para Utilizar TCP e UDP

Na arquitetura da solução proposta, a escolha entre **TCP** (Transmission Control Protocol) e **UDP** (User Datagram Protocol) é estratégica, considerando as características específicas e os requisitos de cada componente.

No componente **Rendezvous Point** utiliza-se o TCP para a troca de informações durante o processo de inicialização da rede. O TCP é escolhido devido à sua confiabilidade na entrega de dados, o que é crucial para garantir uma configuração inicial precisa e sem erros. A criação da rede depende fortemente da correta descoberta de vizinhos e servidores, tornando o TCP uma escolha apropriada para essa fase crítica.

No **Nodo**, a escolha entre TCP e UDP é orientada pela natureza das comunicações. O TCP é empregado para a criação e gestão da rede *overlay*, garantindo uma comunicação confiável e ordenada entre os nós. A confiabilidade é essencial para a integridade da estrutura da rede. Por outro lado, o UDP é utilizado na comunicação com o **Servidor**, o **Cliente** e outros **Nodos** para a distribuição da *stream*. A natureza sem conexão do UDP é adequada para a transmissão eficiente de dados em tempo real, onde a prioridade é a velocidade e a latência, e alguma perda de pacotes é tolerável.

No **Servidor** utiliza-se o UDP para fornecer a *stream* ao **Rendezvous Point**. Neste contexto, o UDP é preferido devido à sua eficiência na transmissão de dados em alta velocidade, sendo uma escolha adequada para aplicações que demandam entrega rápida, como o *streaming* de dados.

O **Cliente**, que é a aplicação do cliente responsável por receber a *stream*, utiliza o UDP para a receção eficiente dos dados, novamente enfatizando a importância da velocidade e latência nesse

contexto.

No componente **BootStrapper**, a opção pelo TCP é fundamentada na necessidade de uma comunicação confiável durante o processo inicial de arranque da rede. Dada a sua fiabilidade na entrega de dados, o TCP é essencial para assegurar uma configuração precisa e sem erros durante a descoberta inicial de vizinhos e servidores. O **BootStrapper**, como ponto de partida na rede, desempenha um papel crucial na facilitação dessa descoberta, justificando assim a escolha do TCP para esta fase crítica. Esta abordagem assegura uma base sólida para a criação e estabelecimento eficaz da rede *overlay*.

Em suma, a escolha entre TCP e UDP na arquitetura da solução é guiada pelas necessidades específicas de cada componente, garantindo a eficiência e a confiabilidade nas áreas onde são mais cruciais.

Razões para Dividir os Componentes

A opção por dividir a arquitetura da solução em **cinco componentes distintos**, em vez de adotar uma abordagem onde um servidor seria configurado como um nodo com mais permissões e o cliente seria semelhante, baseia-se em várias considerações estratégicas e funcionais.

As principais razões para esta divisão incluem:

1. Escalabilidade e Desempenho:

A divisão em cinco componentes permite uma escalabilidade independente de cada função, facilitando o ajuste de recursos caso seja necessário.

2. Modularidade e Manutenção:

Cada componente desempenha um papel específico, facilitando a manutenção e atualizações individuais sem afetar outros componentes.

3. Adaptabilidade a Requisitos Específicos:

A escolha de protocolos de comunicação diferentes, como TCP e UDP, atende às necessidades específicas de cada componente, como inicialização confiável e transmissão eficiente de *stream*.

4. Resiliência e Redundância:

A divisão em componentes permite a implementação de estratégias de resiliência e redundância de forma granular, garantindo que a falha em um componente não afete outros, como, por exemplo, no caso do Servidor.

Capítulo 3

Estrutura do Projeto

Tal como é possível ver na figura seguinte, o nosso projeto está dividido em três pastas:

1. **Documents**
2. **Source**
3. **Topologias**

```
core@xubuncore:~/Desktop/ESR-main$ tree
.
├── Documents
│   ├── ESR-Enunciado-TP2-2023-v1.pdf
│   ├── ESR.vpp
│   ├── README.md
│   └── relatório.pdf
├── Source
│   ├── bootstrap.py
│   ├── bootstrap.txt
│   ├── bsNode.py
│   ├── ClienteGUI.py
│   ├── flooding_utils.py
│   ├── Node.py
│   ├── oClientGUI.py
│   ├── oNode.py
│   ├── oRP.py
│   ├── oServer.py
│   ├── README.md
│   ├── RendezvousPoint.py
│   ├── routing_utils.py
│   ├── server.py
│   ├── utils.py
│   └── video.mp4
└── Topologias
    ├── ESR-Teste1.imn
    ├── ESR-Teste2.imn
    ├── ESR-Teste3.imn
    ├── ESR-Teste4.imn
    ├── ESR-Teste5.imn
    ├── README.md
    └── test1.imn

3 directories, 28 files
```

Figura 3.1: Estrutura do Projeto

Na primeira pasta, **Documents**, tal como o próprio nome sugere, encontram-se todos os documentos, tais como o relatório, o enunciado e o ficheiro `.vpp` usado na realização de diversas figuras para o relatório.

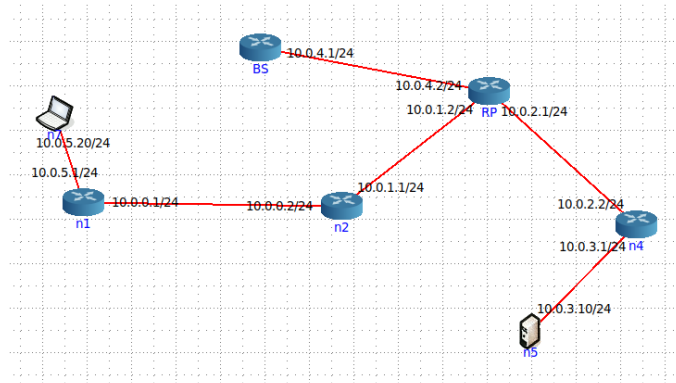


Figura 3.2: Vídeo da Stream

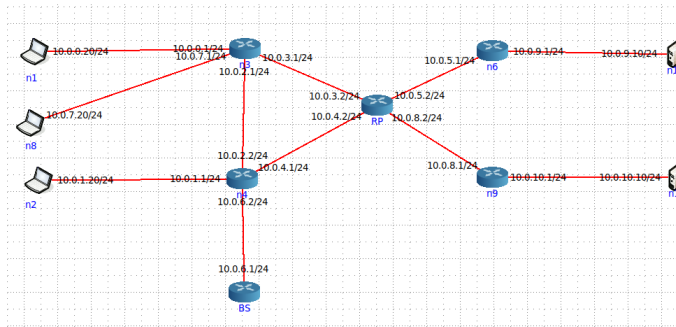
Na segunda pasta, **Source**, encontra-se todo o código realizado no projeto, bem como, o vídeo usado.

Por fim, na última pasta, **Topologias**, estão armazenadas todas as topologias criadas e testadas pelo grupo ao longo de todo o projeto, com o intuito de visualizar todas as etapas solicitadas.

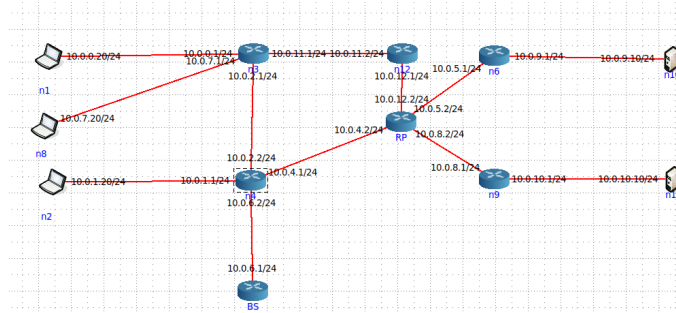
Algumas das topologias usadas foram:



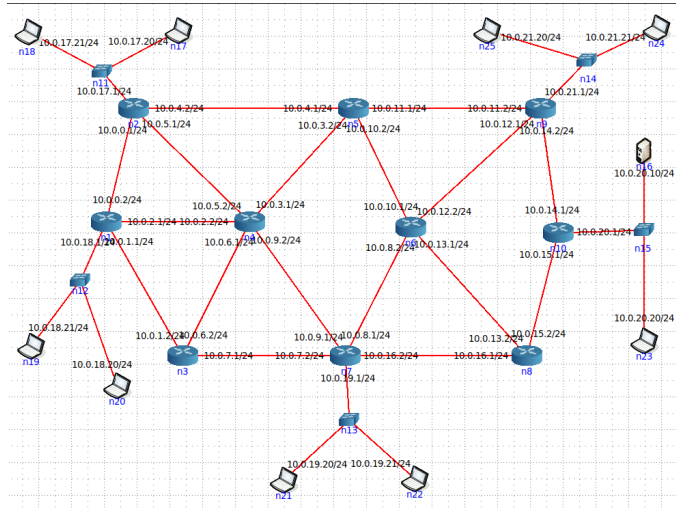
(a) Topologia 1



(b) Topologia 2



(c) Topologia 3



Capítulo 4

Especificação dos Protocolos

Formato das Mensagens Protocolares

Existem três tipos de mensagens protocolares:

- **F** - Flood
- **SA** - Still Alive
- **STR** - Stream
- **P** - Play

Flood

As mensagens do tipo **Flood** servem para, como o próprio nome indica, fazer o *flooding* da rede, isto é, para que cada Nodo, saiba quais os nodos vizinhos a que se pode conectar.

Este tipo de mensagem tem a seguinte estrutura:

FLAGS	IP	NÚMERO DE SALTOS	LATENCIA	TIMESTAMP
-------	----	---------------------	----------	-----------

Figura 4.1: Estrutura de uma mensagem do tipo *Flood*

As possíveis *flags* para o **Nodo** são:

F	UN	R	Resposta ao pedido do Nodo fornecedor
	O nodo só tem um elemento na <i>routing table</i>	Default	Pedido do Nodo fornecedor
	R	Resposta ao pedido de Flooding	
	Default	Pedido de Flooding	

Figura 4.2: Flags de uma mensagem do tipo *Flood* do Nodo

Still Alive

Após fazer o *flooding* da rede, é necessário fazer a monitorização da rede, usando as mensagens do tipo **Still Alive**, estas mensagens são enviadas de 20 em 20 segundos de maneira a perceber se o nodo ao qual estamos ligados ainda está ativo e obter informações sobre a qualidade da ligação.

Este tipo de mensagens tem a seguinte estrutura:

SA	IP	TIMESTAMP	LATENCIA DO NODO QUE ENVIOU A MENSAGEM
----	----	-----------	--

Figura 4.3: Estrutura de uma mensagem do tipo *Still Alive*

Stream

Um tópico que falta falar é a construção das rotas para a entrega de dados. É nesta fase que são usados os pacotes **Stream**. Estas mensagens podem ser de dois formatos: **ascendente** ou **descendente**.

As mensagens do tipo ascendente vão do **Client** para o *Rendezvous Point* registando a rota mais rápida, enquanto que as do tipo descendente vão do *Rendezvous Point* para o **Client** pela rota mais rápida previamente calculada.

Este tipo de mensagens tem a seguinte estrutura:

Flags	Nº de lps	ip_client	ip_node1	ip_node2	...
-------	-----------	-----------	----------	----------	-----

Figura 4.4: Estrutura de uma Mensagem do tipo *Stream* para o RP

Para o caso do **RP**, **Cliente** e **Nodo** este tem as seguintes *flags*:

STR	C	Percorre a topologia ascendentemente
	D	Percorre a rota calculada de forma descendente
	S	Pedido para parar a Stream

Figura 4.5: Flags de uma Mensagem do tipo *Stream* para o RP, Cliente e Nodo

Por outro lado, para o caso do **Server**, este tem as seguintes *flags*:

STR	C	Percorre a topologia ascendentemente
	S	Pedido para parar a Stream

Figura 4.6: Flags de uma Mensagem do tipo *Stream* para o Server

Play

Por fim, o pacote **Play**. Este pacote é usado para o Servidor transmitir, por *unicast*, para o RP a *stream*.

Este tipo de mensagens tem a seguinte estrutura:

P	Pedido para iniciar a transmitir a Stream
---	---

Figura 4.7: Estrutura de uma mensagem do tipo *Play*

Diagramas de Sequência de Mensagens

De seguida, com recurso a Diagramas de Sequência, iremos retratar como são efetuadas todas as mensagens protocolares presentes no nosso projeto, bem como, entre que componentes é feita essa troca.

Diagrama de Pedido de Flood

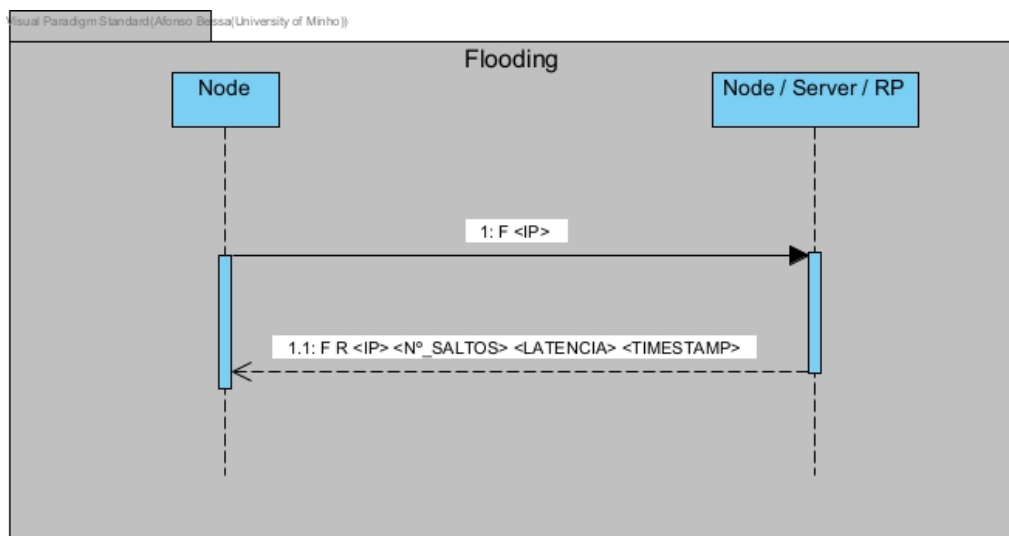


Figura 4.8: Diagrama de Pedido de Flood

4.0.1 Diagrama de Pedido de Flood para Ligações Unique Way

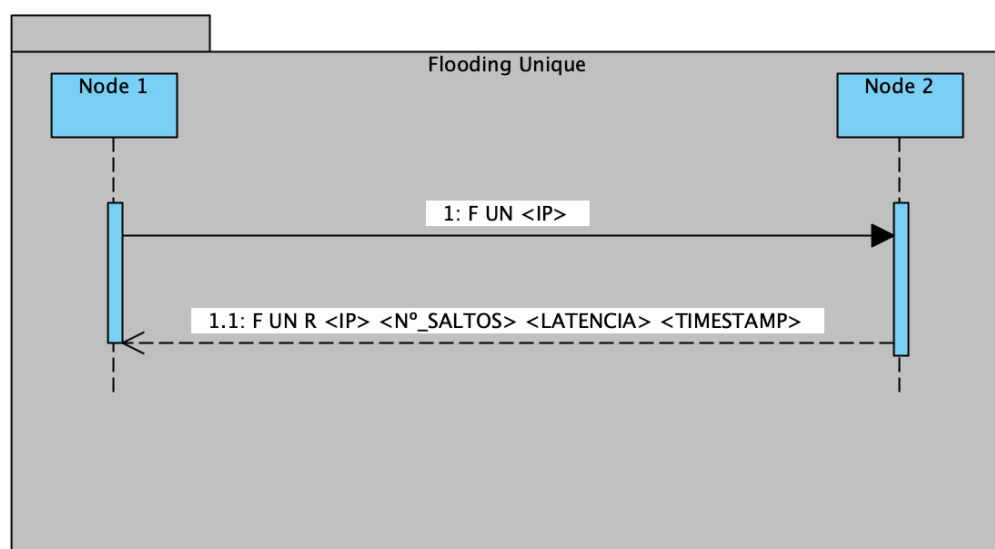


Figura 4.9: Diagrama de Pedido de Flood para ligações Unique Way

4.0.2 Diagrama de Stream

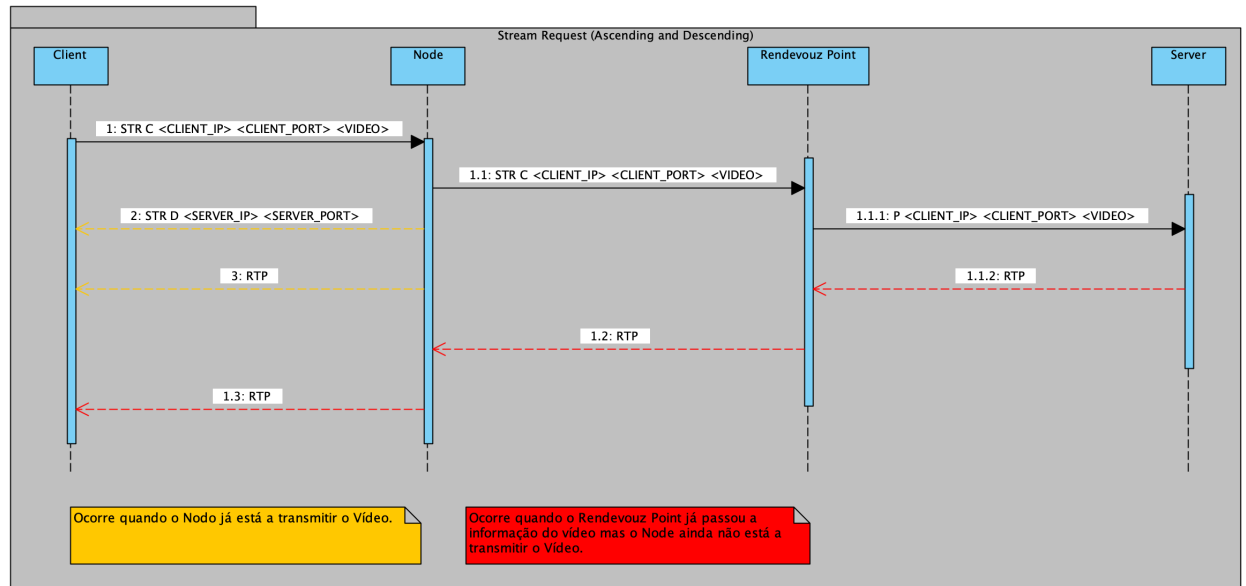


Figura 4.10: Diagrama de Stream

4.0.3 Diagrama de Paragem da Stream

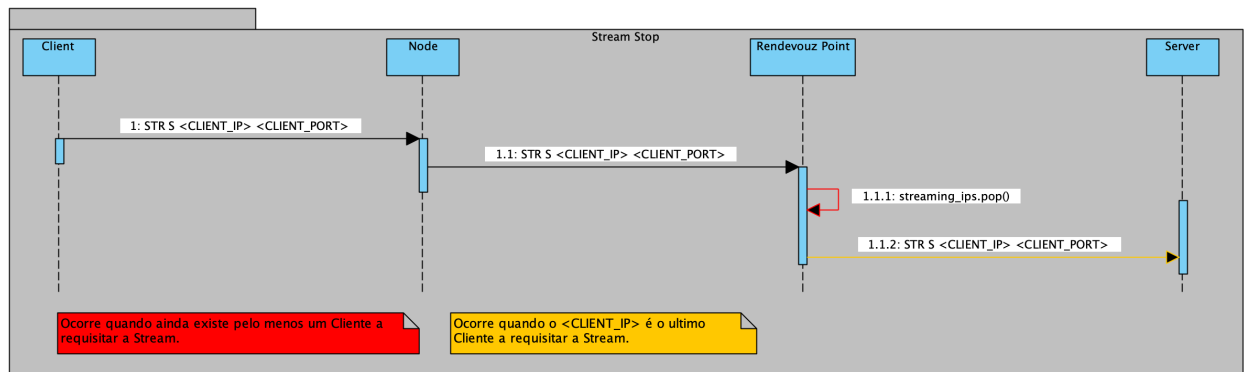


Figura 4.11: Diagrama da Paragem da Stream

4.0.4 Diagrama do Still Alive

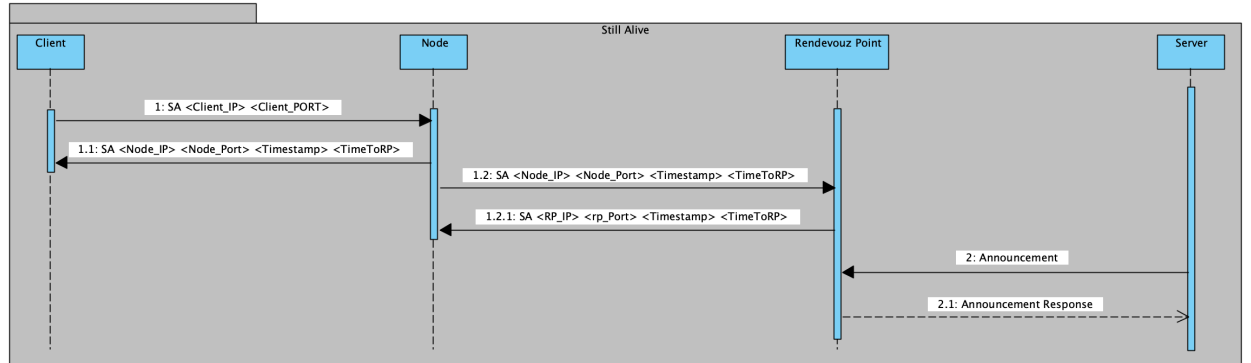


Figura 4.12: Diagrama do Still Alive

Capítulo 5

Implementação

A implementação desenvolvida está dividida em duas partes:

1. **Criação** e **gestão** da rede *overlay*;
2. **Criação** de rotas, **gestão** de pacotes e **distribuição** do *streaming* de vídeo.

Neste projeto foram utilizadas diversas bibliotecas para nos ajudarem a realizar algumas tarefas.

- A biblioteca **cv2** foi utilizada para ler as *frames* do vídeo para o *streaming*.
- O **PIL**, *Python Imaging Library*, foi utilizado para apresentar as imagens na *GUI*.
- As bibliotecas *threading* e *_thread* foram utilizadas para lançar as várias *threads*.
- Recorremos ao *numpy* para fazer o *encoding* e, posteriormente, *decoding* das imagens para *bytes* e vice-versa.
- A *imutils* foi utilizada para fazer o *resize* das *frames* do vídeo lidas antes de serem transmitidas.
- Finalmente, o *tkinter* foi utilizado para criar a *GUI*.

Rede Overlay

A rede overlay é constituída pelos **nodos** e por um **bootstrapper**.

BoostStrapper

O **bootstrapper** serve para dar a conhecer aos nodos quais os seus vizinhos, tal como foi mencionado anteriormente. Esta distribuição está feita previamente e é definida num ficheiro de texto (**bootstrap.txt**). Este ficheiro que é lido pelo *bootstrapper* quando é iniciado e é guardada em memória quais os vizinhos de cada nodo. Cada nodo é composto pelo seu *ip* e *port*, separados por ':' (exemplo: 10.0.1.1:2000).

As figuras 5.1, 5.2 e 5.3 apresentam a estrutura, exemplo e resultado da constituição e execução do ficheiro *bootstrapper*:

```
<Nodo1>
<Nodo1_Vizinho1> <Nodo1_Vizinho2> <Nodo1_Vizinho3>
<Nodo1>
<Nodo2_Vizinho1> <Nodo2_Vizinho2>
<Nodo3>
<Nodo3_Vizinho1> <Nodo3_Vizinho2>
```

Figura 5.1: Exemplo da Estrutura do *bootstrap.txt*

```
10.0.0.20:2000
10.0.0.1:2000
10.0.0.1:2000
10.0.0.20:2000 10.0.1.1:2000 10.0.3.2:2000 10.0.1.1:2000
10.0.1.20:2000
10.0.1.1:2000
10.0.1.1:2000
10.0.1.20:2000 10.0.6.1:2000 10.0.3.2:2000 10.0.0.1:2000
10.0.6.1:2000
10.0.1.1:2000
10.0.3.2:2000
10.0.1.1:2000 10.0.0.1:2000 10.0.5.1:2000 10.0.8.1:2000
10.0.0.20:2000
10.0.0.1:2000
10.0.5.1:2000
10.0.3.2:2000 10.0.9.10:2000
10.0.8.1:2000
10.0.3.2:2000 10.0.10.10:2000
10.0.9.10:2000
10.0.5.1:2000
10.0.10.10:2000
10.0.8.1:2000
```

Figura 5.2: Exemplo do Ficheiro *bootstrap.txt*

```
File bootstrap.txt
["10.0.0.20:2000\n", "10.0.0.1:2000\n", "10.0.0.1:2000\n", "10.0.0.20:2000 10.0.1.1:2000 10.0.3.2:2000 10.0.1.1:2000\n", "10.0.1.20:2000\n", "10.0.1.1:2000\n", "10.0.1.1:2000\n", "10.0.1.20:2000 10.0.6.1:2000 10.0.3.2:2000 10.0.0.1:2000\n", "10.0.6.1:2000\n", "10.0.1.1:2000\n", "10.0.3.2:2000\n", "10.0.1.1:2000 10.0.0.1:2000 10.0.5.1:2000 10.0.8.1:2000\n", "10.0.0.20:2000\n", "10.0.0.1:2000\n", "10.0.5.1:2000\n", "10.0.3.2:2000 10.0.9.10:2000\n", "10.0.8.1:2000\n", "10.0.3.2:2000 10.0.10.10:2000\n", "10.0.9.10:2000\n", "10.0.5.1:2000\n", "10.0.10.10:2000\n", "10.0.8.1:2000"]
Length of the File
22
Dictionary with Neighbors
{"10.0.0.20:2000": [{"10.0.0.1": "2000"}], "10.0.0.1:2000": [{"10.0.0.20": "2000"}, {"10.0.1.1": "2000"}, {"10.0.3.2": "2000"}, {"10.0.1.1": "2000"}], "10.0.1.20:2000": [{"10.0.1.1": "2000"}], "10.0.1.1:2000": [{"10.0.1.20": "2000"}, {"10.0.6.1": "2000"}, {"10.0.3.2": "2000"}, {"10.0.0.1": "2000"}], "10.0.6.1:2000": [{"10.0.1.1": "2000"}, {"10.0.3.2:2000": [{"10.0.1.1": "2000"}, {"10.0.0.1": "2000"}, {"10.0.5.1:2000": [{"10.0.3.2": "2000"}, {"10.0.9.10": "2000"}], "10.0.8.1:2000": [{"10.0.3.2": "2000"}, {"10.0.10.10:2000": [{"10.0.8.1": "2000"}]}]}
```

Figura 5.3: Resultado da Execução do *bootstrap.txt*

Flooding

Para construir a rede *overlay* foi utilizada uma técnica de **flood dinâmico**. Primeiramente, cada nodo ao iniciar faz um pedido ao *bootstrapper* para obter os seus vizinhos. De seguida, caso um Cliente se ligue à rede inicia-se o *flooding*, ou seja, os nodos vão se dando a conhecer uns aos outros e ao mesmo tempo vão registando quais são os caminhos mais rápidos até ao RP.

Caso estes estejam desligados, o nodo vai continuar a tentar enviar-lhes um pedido a cada 20 segundos. A escolha dos 20 segundos para os pedidos de *flooding* entre nodos deve-se a razões práticas para equilibrar eficiência na comunicação, controle de tráfego e minimização do impacto na rede. Esse intervalo evita congestionamentos ao espaçar os pedidos, garantindo tempo suficiente para processamento e resposta dos nodos, sem comprometer a estabilidade do sistema. Ao manter esse intervalo, otimizamos a eficiência operacional, evitando oscilações rápidas na carga de tráfego

e proporcionamos um tempo adequado para lidar com situações em que os vizinhos podem estar temporariamente indisponíveis.

Para cada vizinho, é enviado um pacote de pedido de *flood*, ao qual o vizinho vai responder com o melhor tempo até ao RP e o número de saltos associado. Ao receber esses dados, o nodo vai colocar na sua *routing table* o endereço do vizinho, seguido do número de saltos até ao RP, a ordem de grandeza do tempo até ao RP, uma *flag* a sinalizar se é um nodo de *backup* ou não, e o *timestamp* do tempo até ao RP. Após adicionar o nodo à *routing table*, de maneira a garantir que o primeiro vizinho da *routing table* é sempre o mais rápido até ao RP, esta é ordenada pela seguinte maneira:

1. Base na *flag* de *backup*;
2. Pela ordem de grandeza do tempo até ao RP;
3. Pelo número de saltos até ao RP.

Unique Way Backups

Caso um nodo apenas tenha um vizinho conectado após 15 segundos de ter iniciado, este vai enviar uma mensagem ao seu vizinho a pedir o seu fornecedor. Após receber a resposta, este nodo é adicionado à *routing table*, sendo marcado como *backup*. Os nodos *backup* apenas são usados caso nenhum dos vizinhos diretos esteja disponível. Desta forma é possível prevenir que o nodo perca o contacto com a rede caso o seu único vizinho fique indisponível, considerando que esta seja uma rede *self-healing*.

Still Alive

É necessário que os nodos verifiquem a disponibilidade dos seus vizinhos, e para isso foi implementado um sistema de *still alive*. Para isto, os nodos trocam mensagens de *still alive* entre si, de 20 em 20 segundos, notificando não só a sua disponibilidade, como também atualizam o tempo até ao servidor atual. Desta forma, como *routing table* é sempre reordenada quando é atualizada, é possível detetar quando a ligação entre certos nodos aumenta o que posteriormente será usado para fazer desvios nas rotas da *stream* de vídeo ou pelo contrário atualizar rotas que melhores para a transmissão da *stream*.

Através deste *flood* dinâmico, conseguimos lidar facilmente com nodos a entrarem e a saírem da rede, e com o tratamento dos vizinhos únicos conseguimos que a rede seja *self-healing*. Através das mensagens de *still alive*, conseguimos ir também atualizando as *routing tables*, conseguindo assim detetar facilmente atrasos em ligações a certos nodos.

A abordagem adotada baseou-se na seguinte ideia: ao verificar se o endereço de origem do pacote está presente na lista de IPs da *routing table*, avaliamos a latência da conexão. A latência é calculada com base nas informações temporais e de sequência do pacote, juntamente com considerações sobre o *packet loss*. Uma vez identificada uma alteração significativa na latência entre os nós, a *routing table* é atualizada, promovendo uma adaptação dinâmica da rede.

Adicionalmente, integramos a capacidade de gerir a interrupção e retoma dos fluxos de vídeo, respondendo a eventos específicos relacionados aos nodos na *routing table*. Este procedimento não apenas fomenta a estabilidade e eficácia da rede, mas também evidencia a capacidade de realizar ajustes automáticos em resposta às alterações nas condições da rede.

Streaming de Vídeo

O Streaming de Vídeo é constituído por quatro grandes componentes: **RP**, **Nodo**, **Cliente** e **Servidor**.

RP

O **Rendezvous Point** é capaz de receber e retransmitir pacotes UDP para outros nodos interessados na transmissão. Além disso, o nodo lida com pacotes TCP relacionados à transmissão de *streaming*, como pacotes de estabelecimento e paragem de *stream*. A *routing table* é atualizada através do *announcement* feito pelos servidores quando se ligam à rede, para partilhar informações sobre latência e disponibilidade de serviços de *streaming*. Essa tabela é fundamental para direcionar pacotes de *streaming* aos nodos adequados na rede. Esta *routing table* ao contrário da dos nodos tem informações sobre os servidores disponíveis.

Nodo

No **Nodo**, a gestão dos pacotes de transmissão de *streaming* (**STR**) é realizada por métodos específicos. Quando um nodo recebe um pacote **STR ascendente** (STR C), verifica se já está a transmitir, ou seja, se já tem a *stream* pedida pelo Cliente. Se não estiver, envia o pacote de pedido para os seus vizinhos e adiciona o seu próprio endereço à lista de destinatários. Se estiver a transmitir, adiciona o endereço do nodo de origem à lista de destinatários. Por outro lado, ao receber um pacote **STR descendente** (STR D), acrescenta o endereço do nodo de origem à lista de IPs de *streaming* e retransmite o pacote para outros destinatários, mantendo assim a transmissão em "cascata". O nodo também é responsável por gerir o término da transmissão, removendo endereços da lista de destinatários ao receber um pacote **STR stop** (STR S), e se a lista estiver vazia, envia uma mensagem ao nodo de origem, ou seja, o RP, para interromper o envio da transmissão. Quando o RP não tem mais clientes a pedir a stream envia um pedido ao Servidor para este para o envio da stream.

Em suma, os Nodos seguem uma lógica de transmissão de *streaming* em "cascata", tal como foi mencionado anteriormente, onde cada nodo retransmite a transmissão para os seus vizinhos, assegurando uma distribuição eficiente do conteúdo na rede. O uso de pacotes STR, juntamente com a lógica de *flooding* e gestão dinâmica de destinatários, permite que a transmissão se adapte a qualquer alteração na topologia da rede.

Cliente

Nesta classe, os pacotes **STR** são utilizados para estabelecer, controlar e encerrar a transmissão de vídeo. Quando o utilizador clica no botão **Começar stream**, é enviado um pedido **STR C** para o servidor. Este pedido contém informações sobre o cliente, incluindo o endereço IP e porta do cliente e o nome do vídeo a ser transmitido. O servidor responde enviando pacotes de vídeo através de pacotes UDP.

Por outro lado, ao clicar no botão **Sair**, é enviado um pedido **STR S** ao servidor, indicando que o cliente deseja interromper a transmissão. Este pedido é enviado para o servidor, que interrompe o envio de pacotes de vídeo ao cliente.

Para além disso, é possível confirmar que a transmissão está a ocorrer (**STR D**), que indicam

que um dos nodos já tem a stream. Estes métodos são utilizados para atualizar o estado da aplicação e a interface do utilizador. A comunicação entre o cliente e o nodo a que está ligado é realizada através de sockets TCP e UDP.

Server

A classe **Server** é responsável por gerir a transmissão de vídeo e os pacotes de interação com o RP. Ela trata do envio de pacotes de vídeo através de UDP e do processamento de pacotes recebidos, gerenciando a inicialização e paragem do *streaming*. Estas funcionalidades são essenciais para garantir a eficiência da transmissão de vídeo e otimizar a comunicação na rede.

Em resumo, a lógica subjacente a este processo envolve a codificação e decodificação de pacotes RTP para a transmissão de vídeo, assim como o tratamento de pacotes específicos, como **STR** para controlo de *streaming*. O programa implementa também uma funcionalidade de **announcement** para indicar a continuidade da presença do Servidor ao RP.

Capítulo 6

Testes e Resultados

De seguida iremos mostrar todas as funcionalidades implementadas no Projeto, tais como:

- Flood
- Estabelecimento de Rota
- Adição de Melhor Rota
- Paragem da Stream
- Servidores Dinâmicos
- Adição e Remoção de Nodos

Para isso, iremos utilizar a seguinte topologia:

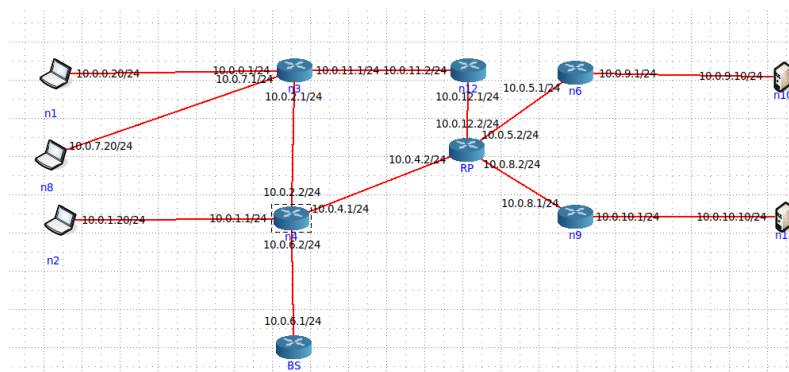
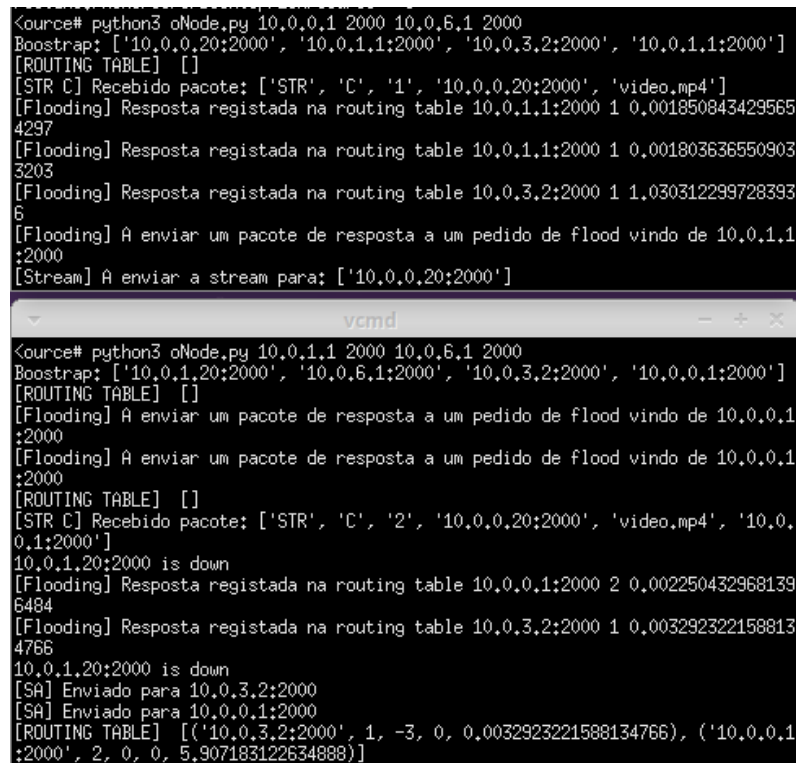


Figura 6.1: Topologia

Para iniciar os diferentes componentes do sistema, é necessário obedecer à seguinte sintaxe:

- **Bootstrapper:** python3 bsNode.py <ip_bootstrapper ><port_bootstrapper>
- **RP:** python3 oRP.py <ip_RP ><port_RP><ip_bootstrapper ><port_bootstrapper>
- **Nodos:** python3 oNode.py <ip_nodo><port_nodo><ip_bootstrapper><port_bootstrapper>
- **Server:** python3 oServer.py <ip_server><port_server><ip_bootstrapper ><port_bootstrapper><ip_RP ><port_RP>
- **Cliente:** python3 oClientGUI.py <ip_cliente><port_cliente><ip_nodo><port_nodo>

Flood



```
<source# python3 oNode.py 10.0.0.1 2000 10.0.6.1 2000
Bootstrap: ['10.0.0.20:2000', '10.0.1.1:2000', '10.0.3.2:2000', '10.0.1.1:2000']
[ROUTING TABLE] []
[STR C] Recebido pacote: ['STR', 'C', '1', '10.0.0.20:2000', 'video.mp4']
[Flooding] Resposta registrada na routing table 10.0.1.1:2000 1 0.001850843429565
4297
[Flooding] Resposta registrada na routing table 10.0.1.1:2000 1 0.001803636550903
3203
[Flooding] Resposta registrada na routing table 10.0.3.2:2000 1 1.030312299728393
6
[Flooding] A enviar um pacote de resposta a um pedido de flood vindo de 10.0.1.1
:2000
[Stream] A enviar a stream para: ['10.0.0.20:2000']

vcmd
<source# python3 oNode.py 10.0.1.1 2000 10.0.6.1 2000
Bootstrap: ['10.0.1.20:2000', '10.0.6.1:2000', '10.0.3.2:2000', '10.0.0.1:2000']
[ROUTING TABLE] []
[Flooding] A enviar um pacote de resposta a um pedido de flood vindo de 10.0.0.1
:2000
[Flooding] A enviar um pacote de resposta a um pedido de flood vindo de 10.0.0.1
:2000
[ROUTING TABLE] []
[STR C] Recebido pacote: ['STR', 'C', '2', '10.0.0.20:2000', 'video.mp4', '10.0.
0.1:2000']
10.0.1.20:2000 is down
[Flooding] Resposta registrada na routing table 10.0.0.1:2000 2 0.002250432968139
6484
[Flooding] Resposta registrada na routing table 10.0.3.2:2000 1 0.003292322158813
4766
10.0.1.20:2000 is down
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.0.1:2000
[ROUTING TABLE] [(('10.0.3.2:2000', 1, -3, 0, 0.0032923221588134766), ('10.0.0.1
:2000', 2, 0, 0, 5.907183122634888))]
```

Figura 6.2: Flood só com Nodos - a usar: n3 (cima) e n4 (baixo)

Nesta figura, podemos observar que, quando os nodos são iniciados, nenhum deles tem acesso ao RP. Embora se reconheçam (como comprovado pela ausência da mensagem "ip is down"), não partilham informação devido à sua inexistência.

Funcionamento Geral

Estabelecimento de Rota

Primeiramente, o Servidor anuncia a sua chegada à rede ao RP quando é inicializado. Isto permite que o RP tenha informação sobre os serviços de *stream* fornecidos pelo Servidor e que este saiba que o RP está disponível para receber a *stream* quando um Cliente a pedir. Deste modo, conseguimos ligar um Servidor a qualquer momento à rede de forma dinâmica, o que permite depois que o RP escolha o Servidor mais indicado para qualquer pedido de *stream*.

```
</Source# python3 oRP.py 10.0.3.2 2000 10.0.6.1 2000
Bootstrap: ['10.0.1.1:2000', '10.0.0.1:2000', '10.0.5.1:2000', '10.0.8.1:2000', '
']
[RP] Novo serviço de Stream Registrado
<Server.py 10.0.9.10 2000 10.0.6.1 2000 10.0.3.2 2000
Bootstrap: ['10.0.5.1:2000']
Server is listening on 10.0.9.10 2000
[Announcement] Resposta registrada na routing table 10.0.3.2:2000 1 0.001208782196
0449219
```

Figura 6.3: Announcement do Servidor

Quando ligamos um Cliente e pedimos a *stream* este envia um pacote ao Nodo associado. No momento em que o Nodo recebe esse pacote realiza o *flood* de forma a preencher a sua *routing table* e saber qual o melhor caminho até ao RP. Após a conclusão deste processo o nodo propaga a mensagem de pedido da *stream* até ao RP.

Ao receber esse pedido o RP calcula qual o melhor Servidor para servir este cliente e a melhor rota entre a rede *overlay*. De seguida envia pedido ao Servidor escolhido para este iniciar a *stream*, a qual é então propagada até ao cliente da forma mais eficiente.

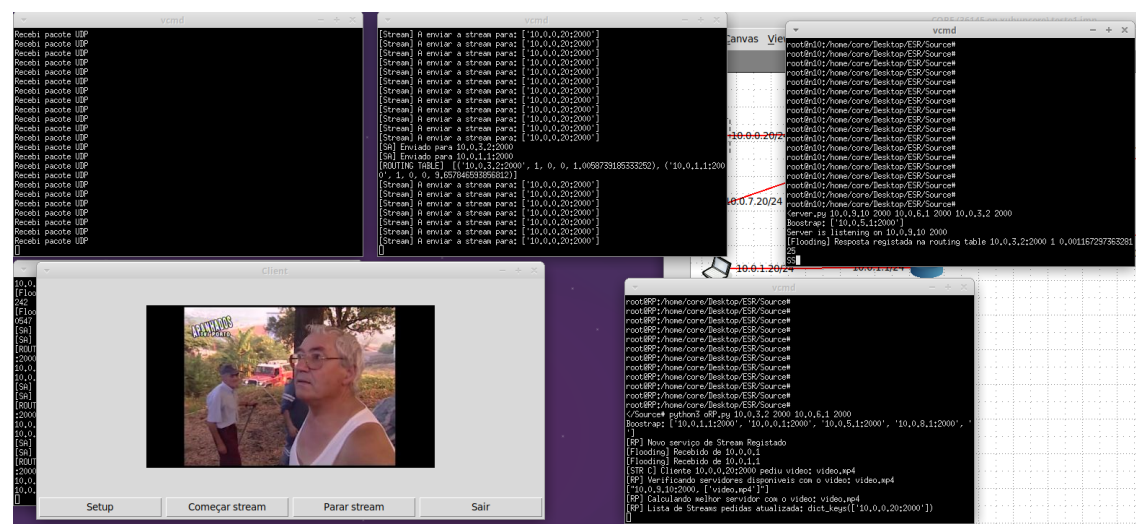


Figura 6.4: Estabelecimento da Melhor Rota

Quando um dos nodos no melhor caminho entre o cliente e o RP já está a receber a stream requisitada, a mensagem de pedido da mesma acaba por não ser transmitida para o RP de modo a evitar fluxos desnecessários e congestionamento da rede. Isto permite uma seleção de rotas de distribuição dinâmicas que se ajustam ao fluxo da rede.

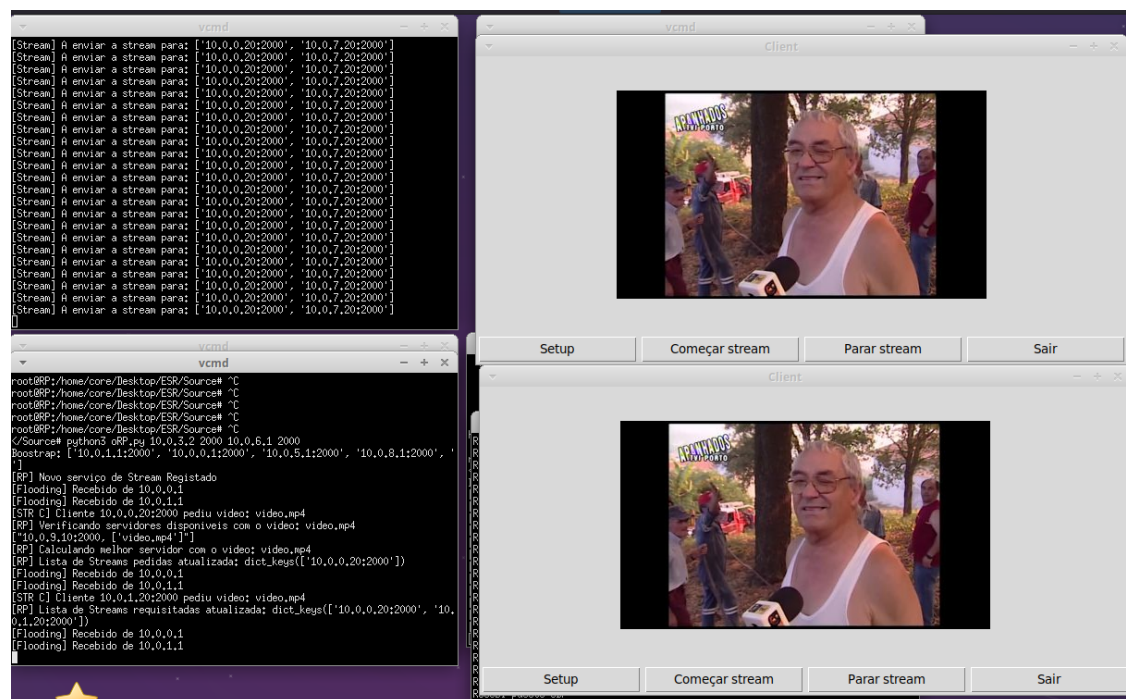


Figura 6.5: Distribuição da Stream pelos Nodos

Devido ao *flooding* dinâmico e às interações entre os elementos da rede, garantimos que a *stream* é distribuída sempre que pelo menos um Servidor está disponível e existe uma rota possível para a distribuir até ao Cliente.

Em consequência dos serviços de monitorização de rede implementados e descritos posteriormente, a rede tem também a capacidade de se ajustar às condições de distribuição da mesma. Assim se uma rota tiver maior latência ou packet loss a rede irá de forma dinâmica ajustar-se a essas condições e calcular uma melhor rota de acordo com essas métricas. Para testar esta funcionalidade o grupo decidiu fazer um pedido de stream de um cliente e verificar qual a sua rota, em seguida aumentar o packet loss de uma das ligações dessa rota e pedir novamente a stream. Consequentemente a rota foi alterada para uma com melhores condições.

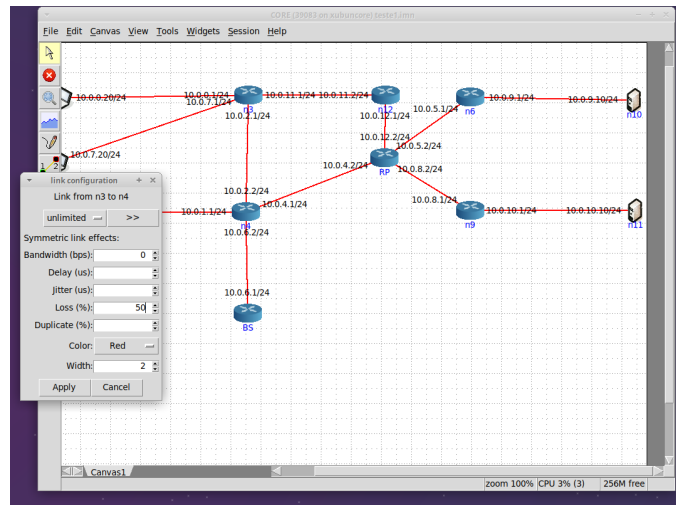


Figura 6.6: Condicionamento da ligação

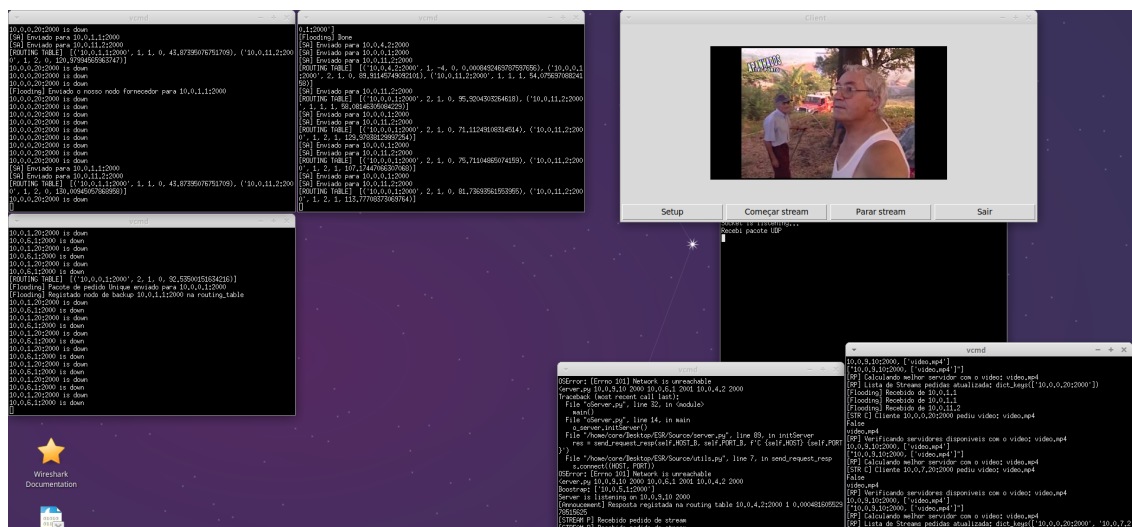


Figura 6.7: Alteração da rota devido ao condicionamento da ligação

No primeiro pedido de stream a rota definida passava por n4, mas depois do condicionamento as routing tables dos nodos alteraram-se e consequentemente a rota do segundo pedido também, fazendo com que a stream fosse distribuída agora por n12.

Paragem da Stream

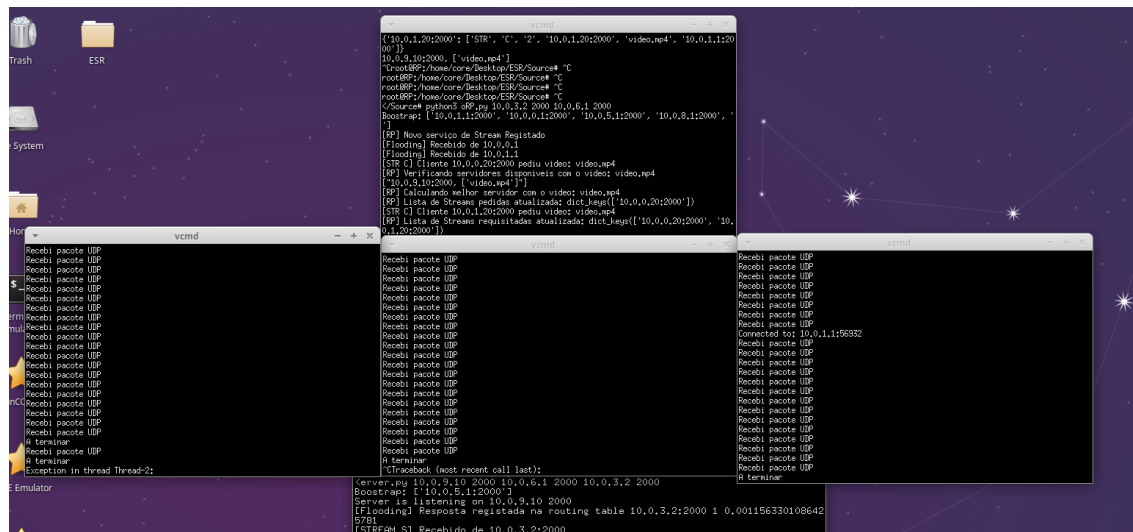


Figura 6.8: Paragem da Stream

No momento em que paramos a *stream*, uma mensagem de paragem é propagada pelo Cliente pela rota até ao RP nos moldes descritos anteriormente. Isto é, no caso de encontrar algum nodo que tem mais alguém a receber o sinal, a mensagem de paragem deixa de ser propagada. Deste modo o RP só sinaliza a paragem de *stream* ao Servidor se não houverem mais clientes a pedir a *stream*. Isto só é possível devido ao facto de os Nodos só propagarem esta mensagem até ao RP se não tiverem mais clientes a pedir a *stream*.

Monitorização da Rede

Still Alive

Com dito em cima, a Monitorização da Rede é feita através de mensagens *Still Alive*, que dão a indicação que determinado Nodo ou Cliente ainda estão vivos atualizando a latência ao RP. Estas mensagens enviam informações sobre a latência, *packet loss* e número de saltos. A latência é calculada usando *timestamps* do momento em que o pacote é recebido e fazendo assim a diferença entre o tempo de envio e o tempo de receção. Isto só é possível porque o **CORE** garante que os relógios de todos os terminais estão sincronizados. Para calcular o *packet loss* cada Nodo calcula quantos pacotes *Still Alive* deveria receber de acordo com os nodos ligados a ele e faz a diferença com os que realmente recebeu. Deste modo, os Nodos combinam estas medições numa só métrica que é usada pela rede para garantir a melhor rota para o serviço de *stream*.

Em funcionamento:

```

vcmd
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.1.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, 0, 0, 1.018066644668579], ('10.0.1.1:2000', 1, 0, 0, 8.080322360992431)]
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000', '10.0.7.20:2000']
[SA] Enviado para 10.0.0.1:2000, '1701627298.4562569', '0.00201463693408203']
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.0.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, -3, 0, 0.0034623146057128906], ('10.0.0.1:2000', 2, 0, 0, 5.943604463239316)]
10.0.1.20:2000 is down

```

Figura 6.9: Monitorização da Rede

Unique Way

Temos também um mecanismo de *self-healing* da rede que permite que um nodo que apenas tenha um nodo ligado a si lhe pergunte pelo seu fornecedor, para que em caso de morte do nodo a si ligado, ele tenha um *backup* para se possa ligar.

```

vcmd
[Stream] A enviar a stream para: ['10.0.0.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000']
[Stream] A enviar a stream para: ['10.0.0.20:2000']
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.1.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, 0, 0, 1.019925594329834], ('10.0.1.1:2000', 1, 0, 0, 5.934389963150024)]
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.1.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, 0, 0, 1.019925594329834], ('10.0.1.1:2000', 1, 0, 0, 8.598205184936523)]
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.1.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, 0, 0, 1.019925594329834], ('10.0.1.1:2000', 1, 1, 0, 11.60269718170166)]
[ROUTING TABLE] [['10.0.1.1:2000', 1, 1, 0, 11.60269718170166)]
[Flooding] Resposta registada na routing table 10.0.3.2:2000 1 0.0012378692626953125
[Flooding] Enviado o nosso nodo fornecedor para 10.0.1.1:2000

vcmd
[SA] Enviado para 10.0.0.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, -3, 0, 0.0029990673065185547], ('10.0.0.1:2000', 2, 0, 0, 4.442081117630005)]
10.0.1.20:2000 is down
10.0.1.20:2000 is down
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.0.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, -3, 0, 0.0029990673065185547], ('10.0.0.1:2000', 2, 0, 0, 7.4493680080413815)]
10.0.1.20:2000 is down
10.0.1.20:2000 is down
[SA] Enviado para 10.0.3.2:2000
[SA] Enviado para 10.0.0.1:2000
[ROUTING TABLE] [['10.0.3.2:2000', 1, -3, 0, 0.0029990673065185547], ('10.0.0.1:2000', 2, 1, 0, 10.45000786781311)]
10.0.1.20:2000 is down
10.0.1.20:2000 is down
[ROUTING TABLE] [['10.0.0.1:2000', 2, 1, 0, 13.454091501235961]]
[Flooding] Pacote de pedido Unique enviado para 10.0.0.1:2000
[Flooding] Registado nodo de backup 10.0.3.2:2000 na routing_table
10.0.1.20:2000 is down
10.0.3.2:2000 is down
~C10.0.1.20:2000 is down
10.0.3.2:2000 is down

```

Figura 6.10: Unique Way

Adição e remoção de nodos da rota

De acordo com os procedimentos anteriormente descritos a nossa rede está preparada para adição e remoção de nodos. Assim se um nodo da melhor rota cair, da próxima vez que a rota for calculada a nossa rede vai ajustar-se a essa perda, bem como o contrário.

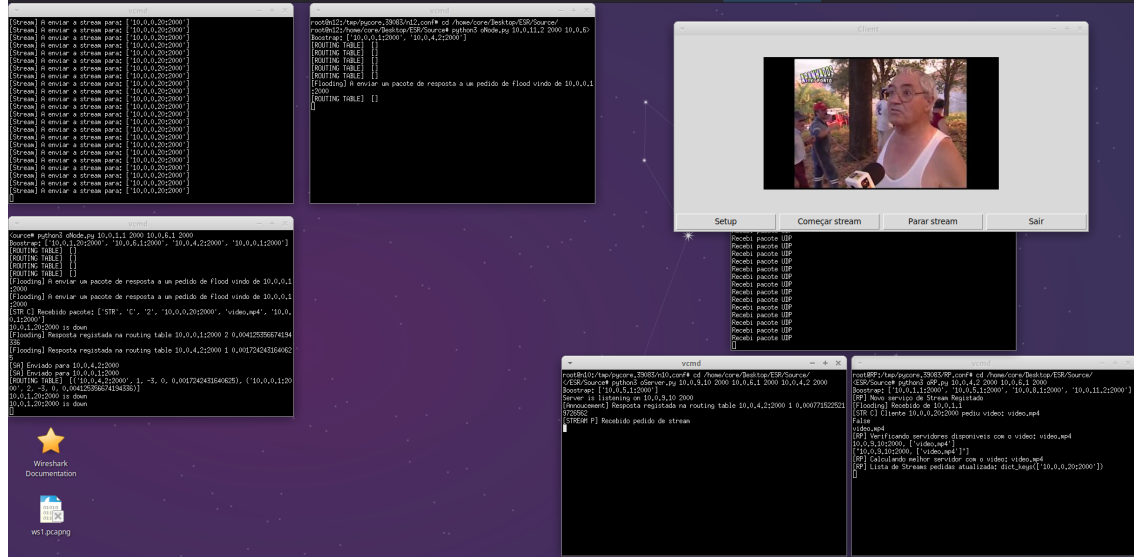


Figura 6.11: Adição e remoção de nodos 1

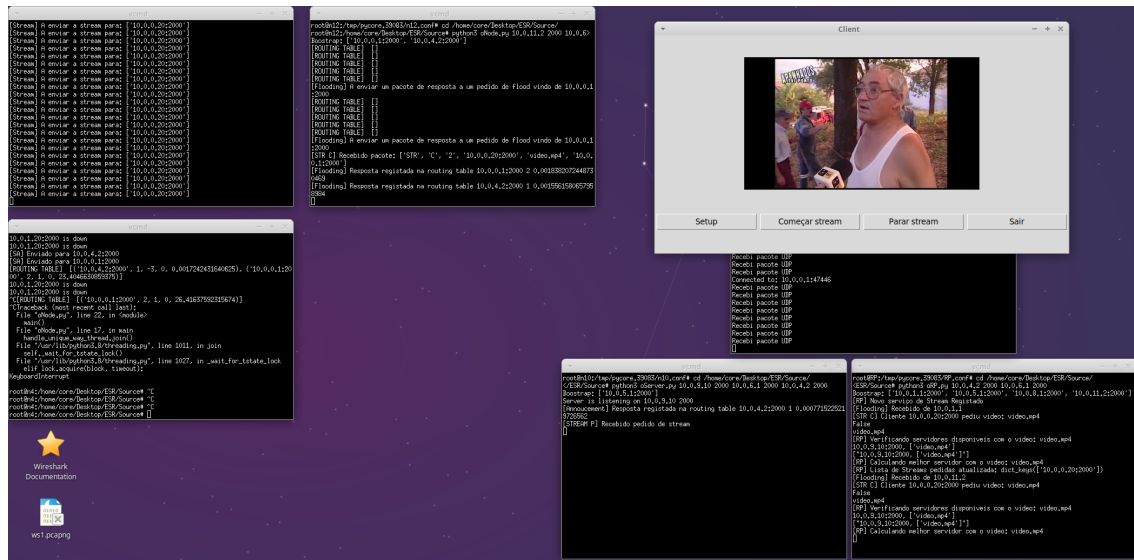


Figura 6.12: Adição e remoção de nodos 1

Capítulo 7

Exemplo de Execução

Através dos dois Diagramas de Sequência seguintes iremos retratar como o **Flooding** acontece, bem como, a **Transmissão de Stream**, de maneira a explicar de forma muito sustinta o que acontece entre os Nodos, Clientes, RP e Server:

Flooding

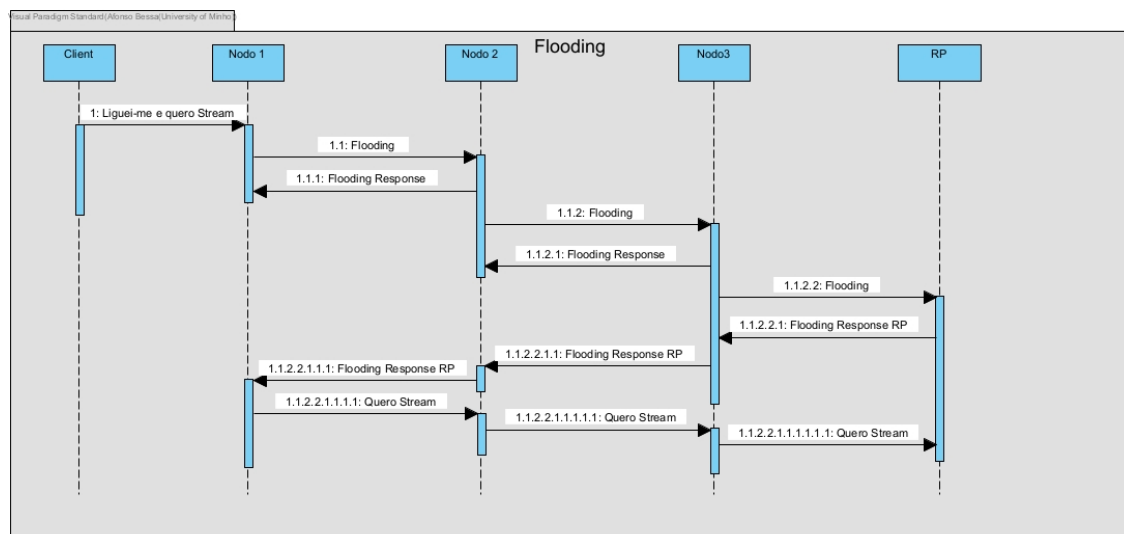


Figura 7.1: Diagrama de Sequência representativo do Flooding

Stream

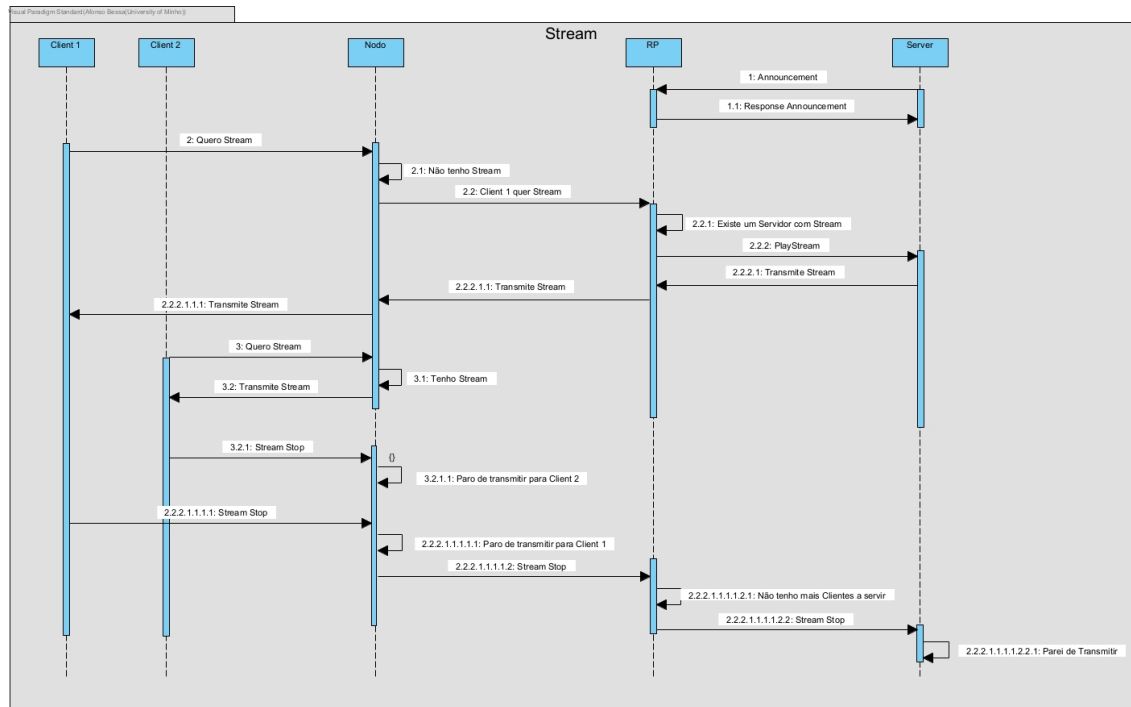


Figura 7.2: Diagrama de Sequência representativo da Stream

Capítulo 8

Conclusões e Trabalho Futuro

Neste projeto, tivemos a oportunidade de desenvolver uma rede *overlay* aplicacional dedicada ao *streaming* de vídeo em tempo real. Esta experiência permitiu-nos aprofundar o nosso entendimento sobre o funcionamento dessas redes, assim como identificar as suas vantagens e desvantagens. A implementação do *flood* dinâmico e o tratamento específico dado aos nodos com apenas uma ligação revelaram a capacidade da nossa solução em ser *self-healing*, ou seja, capaz de lidar eficientemente com nodos que deixam de responder ou com a integração de novos nodos na rede.

Destacamos a importância da nossa abordagem à definição da *routing table*, que desempenhou um papel crucial na identificação das ligações mais rápidas em tempo real. As mensagens de *Still Alive* mostraram-se essenciais não apenas para verificar a vitalidade dos nodos, mas também para atualizar continuamente as *routing tables* de todos os nodos com os tempos mais recentes até ao *Rendezvous Point*. A combinação eficaz destes componentes do sistema, juntamente com o algoritmo de criação dinâmica de rotas de distribuição, possibilitou uma adaptação eficiente das rotas, especialmente em situações de atraso em determinadas ligações. Este enfoque dinâmico e adaptativo fortaleceu a resiliência do sistema, contribuindo para uma experiência mais robusta e eficiente na transmissão de *streams* de vídeo em tempo real.

Como perspetiva de desenvolvimentos futuros, identificamos como uma adição potencialmente vantajosa a capacidade do Cliente escolher entre várias *streams* disponíveis para visualização. A implementação desta funcionalidade implicaria a duplicação das *routing tables* e uma extensão considerável do código existente. Essencialmente, seria necessário criar uma estrutura que permitisse a gestão independente de cada vídeo, garantindo assim uma experiência flexível para o utilizador ao selecionar diferentes fontes de conteúdo para visualização. Esta ampliação da capacidade de escolha por parte do Cliente poderia enriquecer significativamente a experiência de utilização da aplicação, proporcionando uma maior personalização e adaptabilidade às preferências individuais dos utilizadores.